

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



GRAPH AND TREE

Oleh:

Farissa

NIM. 2510817120016

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Farissa
NIM : 2510817120016

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL.....	v
SOAL 1 : KONVERSI ADJACENCY LIST KE ADJACENCY MATRIX	1
A. Source Code.....	3
B. Output Program	4
C. Pembahasan.....	4
SOAL 2 : KONVERSI ADJACENCY MATRIX KE ADJACENCY LIST	7
A. Source Code.....	8
B. Output Program	9
C. Pembahasan.....	10
SOAL 3 : BFS: JARAK TERPENDEK KELUAR LABIRIN.....	13
A. Source Code.....	15
B. Output Program	16
C. Pembahasan.....	16
SOAL 4 : DFS: MENGHITUNG SEMUA JALUR KELUAR LABIRIN	19
A. Source Code.....	21
B. Output Program	22
C. Pembahasan.....	22
SOAL 5 : TREE TRAVERSAL	25
A. Source Code.....	27
B. Output Program	29
C. Pembahasan.....	29
TAUTAN GIT	33

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 1	4
Gambar 2 Screenshot Hasil Jawaban Soal 2	9
Gambar 3 Screenshot Hasil Jawaban Soal 3	16
Gambar 4 Screenshot Hasil Jawaban Soal 4	22
Gambar 5 Screenshot Hasil Jawaban Soal 5	29

DAFTAR TABEL

Table 1 Source Code Adjacency List.....	3
Table 2 Source Code Adjacency Matrix	8
Table 3 Source Code BFS Terpendek Labirin	15
Table 4 Source Code DFS Semua Labirin	21
Table 5 Source Code Tree Traversal	27

SOAL 1 : KONVERSI ADJACENCY LIST KE ADJACENCY MATRIX

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge. Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Table 1 Source Code Adjacency List

1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int main() {
5	ios::sync_with_stdio(false);
6	cin.tie(nullptr);
7	
8	int N;
9	cin >> N;
10	
11	vector<char> labels(N);
12	map<char, int> idx;
13	for (int i = 0; i < N; i++) {
14	cin >> labels[i];
15	idx[labels[i]] = i;
16	}
17	
18	vector<vector<int>> mat(N, vector<int>(N, 0));
19	
20	int M;
21	cin >> M;
22	for (int i = 0; i < M; i++) {
23	char u, v;
24	int w;
25	cin >> u >> v >> w;
26	mat[idx[u]][idx[v]] = w;
27	}
28	
29	cout << "Adjacency Matrix:\n";
30	
31	cout << " ";
32	for (int i = 0; i < N; i++) {
33	cout << " " << labels[i];
34	}
35	cout << "\n";
36	
37	for (int i = 0; i < N; i++) {
38	cout << labels[i];
39	for (int j = 0; j < N; j++) {
40	cout << " " << mat[i][j];
41	}
42	cout << "\n";
43	}


```

44 |
45 |     return 0;
46 | }

```

B. Output Program

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> g++ -std=c++17 -o soal1 soal1.cpp
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a>

```

Gambar 1 Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Soal ini meminta kita mengubah representasi graph berarah berbobot dari daftar edge menjadi adjacency matrix. Tujuannya adalah menghasilkan matrix berukuran $N \times N$ di mana setiap elemen $mat[i][j]$ menyimpan bobot edge dari vertex i menuju vertex j , atau 0 jika edge tersebut tidak ada.

Algoritma yang dipilih adalah pemetaan langsung berbasis array dua dimensi. Alasan pemilihan pendekatan ini karena jumlah vertex dibatasi sangat kecil ($N \leq 10$), sehingga matrix berukuran maksimal $10 \times 10 = 100$ sel, yang jauh dari masalah memori maupun waktu. Selain itu, operasi pengisian matrix hanya perlu dilakukan sekali per edge, membuatnya sangat efisien untuk kasus ini.

Proses berpikir dimulai dari memahami bahwa setiap vertex memiliki label karakter, bukan indeks angka. Karena itu, dibutuhkan struktur pemetaan dari karakter ke indeks integer menggunakan `std::map<char, int>`. Setelah pemetaan terbentuk, setiap edge $U \rightarrow V$ dengan bobot W cukup diterjemahkan menjadi $mat[idx[U]][idx[V]] = W$. Vertex yang tidak memiliki edge ke vertex lain otomatis bernilai 0 karena matrix diinisialisasi dengan nol sejak awal.

Analisis kompleksitas waktu solusi ini adalah $O(N^2)$ untuk inisialisasi matrix dan pencetakan output, serta $O(M)$ untuk memproses seluruh edge, sehingga total kompleksitas adalah $O(N^2 + M)$. Mengingat batasan $N \leq 10$ dan $M \leq N \times (N-1) \leq 90$, kompleksitas ini sangat ringan dan dijamin selesai jauh di bawah batas waktu 1 detik. Kompleksitas ruang juga $O(N^2)$ untuk menyimpan matrix, yang dalam kasus terburuk hanya 100 elemen integer.

Kesimpulannya, pendekatan matrix dua dimensi dengan pemetaan karakter ke indeks adalah solusi paling tepat untuk soal ini karena sederhana, efisien, dan langsung sesuai dengan struktur output yang diminta.

Adjacency matrix adalah salah satu cara merepresentasikan graph menggunakan struktur matrix dua dimensi berukuran $N \times N$, di mana N adalah jumlah vertex. Setiap elemen $mat[i][j]$ menyimpan bobot edge dari vertex i menuju vertex j . Jika tidak terdapat edge di antara keduanya, nilainya adalah 0. Representasi ini berbeda dengan adjacency list yang menyimpan hanya edge yang benar-benar ada, adjacency matrix selalu mengalokasikan ruang untuk semua kemungkinan pasangan vertex, sehingga memudahkan pengecekan keberadaan edge dalam waktu $O(1)$.

Teknik yang digunakan pada soal ini adalah pemetaan langsung (direct mapping), yaitu menerjemahkan setiap edge yang diberikan langsung ke posisi yang sesuai pada matrix. Cara kerjanya dimulai dengan membaca seluruh label vertex dan memetakan setiap karakter ke indeks integer menggunakan struktur `std::map<char, int>`. Pemetaan ini diperlukan karena label vertex berupa karakter alfabet, sementara matrix membutuhkan indeks numerik. Setelah pemetaan terbentuk, setiap edge $U \rightarrow V$ dengan bobot W diterjemahkan menjadi $mat[idx[U]][idx[V]] = W$. Seluruh elemen matrix diinisialisasi dengan nilai 0 sejak awal, sehingga pasangan vertex yang tidak memiliki edge tidak perlu ditangani secara eksplisit.

Asumsi yang dibuat dalam solusi ini adalah bahwa graph bersifat berarah (directed), sehingga edge $U \rightarrow V$ tidak otomatis berarti ada edge $V \rightarrow U$. Oleh karena itu hanya satu sel yang diisi per edge, bukan dua. Selain itu diasumsikan tidak ada self-loop dan tidak ada edge duplikat dengan arah yang sama, sesuai dengan batasan soal, sehingga tidak diperlukan penanganan konflik pengisian sel yang sama.

Analisis kompleksitas waktu solusi ini adalah $O(N^2)$ untuk inisialisasi matrix dan pencetakan output, serta $O(M)$ untuk memproses seluruh M edge, sehingga total kompleksitas adalah $O(N^2 + M)$. Kompleksitas ruang adalah $O(N^2)$ untuk menyimpan matrix. Mengingat batasan

$N \leq 10$ dan $M \leq N \times (N-1) \leq 90$, ukuran matrix maksimal hanya $10 \times 10 = 100$ elemen, sehingga solusi ini dijamin selesai jauh di bawah batas waktu 1 detik dan batas memori 64 MB.

Bukti kebenaran solusi ini dapat dilihat dari sifat pemetaan langsung yang digunakan. Karena setiap label vertex dipetakan ke indeks yang unik dan konsisten, dan setiap edge hanya mengisi tepat satu sel pada matrix, tidak ada kemungkinan terjadi tumpang tindih maupun kekosongan yang tidak semestinya. Pada contoh kasus, edge $A \rightarrow B$ dengan bobot 4 mengisi $\text{mat}[0][1] = 4$, edge $A \rightarrow C$ dengan bobot 2 mengisi $\text{mat}[0][2] = 2$, dan seterusnya. Karena graph bersifat berarah, sel $\text{mat}[1][0]$ tetap bernilai 0 meskipun ada edge dari A ke B, yang terbukti konsisten dengan output yang diharapkan.

SOAL 2 : KONVERSI ADJACENCY MATRIX KE ADJACENCY LIST

Time Limit: 1.0 s Memory Limit: 64 MB
--

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B, 4), (C, 2) B -> (D, 7) C -> (B, 1), (E, 6) D -> (C, 3) E -> -

Penjelasan Contoh Kasus

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Table 2 Source Code Adjacency Matrix

1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int main() {
5	ios::sync_with_stdio(false);
6	cin.tie(nullptr);

```

7
8     int N;
9     cin >> N;
10
11     vector<char> labels(N);
12     for (int i = 0; i < N; i++) cin >> labels[i];
13
14     vector<vector<int>> mat(N, vector<int>(N));
15     for (int i = 0; i < N; i++)
16         for (int j = 0; j < N; j++)
17             cin >> mat[i][j];
18
19     cout << "Adjacency List:\n";
20     for (int i = 0; i < N; i++) {
21         cout << labels[i] << " -> ";
22         bool hasEdge = false;
23         for (int j = 0; j < N; j++) {
24             if (mat[i][j] > 0) {
25                 if (hasEdge) cout << ", ";
26                 cout << "(" << labels[j] << ", " <<
mat[i][j] << ") ";
27                 hasEdge = true;
28             }
29         }
30         if (!hasEdge) cout << "-";
31         cout << "\n";
32     }
33
34     return 0;
35 }

```

B. Output Program



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> g++ -std=c++17 -o soal2 soal2.cpp
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a>

```

Gambar 2 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Soal ini merupakan kebalikan dari soal sebelumnya, diberikan adjacency matrix berukuran $N \times N$, kita diminta mengubahnya menjadi adjacency list yang menampilkan untuk setiap vertex, daftar vertex tujuan beserta bobot edge-nya.

Algoritma yang dipilih adalah iterasi baris per baris pada matrix. Alasan pemilihan pendekatan ini karena struktur adjacency list secara alami sesuai dengan cara kita membaca matrix, setiap baris merepresentasikan satu vertex sumber, dan setiap kolom pada baris tersebut merepresentasikan kemungkinan edge ke vertex tujuan. Dengan iterasi sederhana dua lapis, kita bisa langsung mengekstrak informasi yang dibutuhkan tanpa struktur data tambahan yang kompleks.

Proses berpikir dimulai dari memahami bahwa nilai positif pada $mat[i][j]$ berarti ada edge dari vertex i ke vertex j dengan bobot $mat[i][j]$, sedangkan nilai 0 berarti tidak ada edge. Karena itu, untuk setiap vertex i , kita cukup mengiterasi seluruh kolom j dan mencetak pasangan $(label[j], mat[i][j])$ setiap kali nilai $mat[i][j]$ lebih dari nol. Urutan pencetakan sudah otomatis mengikuti urutan label pada input karena kita mengiterasi kolom dari kiri ke kanan. Satu kasus khusus yang perlu ditangani adalah ketika seluruh nilai pada baris i bernilai 0, yang berarti vertex tersebut tidak memiliki edge keluar sama sekali, sehingga dicetak tanda - sebagai gantinya.

Analisis kompleksitas waktu solusi ini adalah $O(N^2)$ karena kita harus mengiterasi seluruh elemen matrix untuk setiap vertex, dan tidak ada cara yang lebih efisien mengingat kita memang perlu memeriksa setiap sel untuk menentukan ada tidaknya edge. Kompleksitas ruang adalah $O(N^2)$ untuk menyimpan matrix. Dengan batasan $N \leq 10$, seluruh proses berjalan sangat cepat dan tidak mendekati batas waktu maupun memori yang diberikan.

Kesimpulannya, iterasi dua lapis pada matrix adalah pendekatan paling langsung dan tepat untuk soal ini. Tidak diperlukan algoritma tambahan karena struktur matrix sendiri sudah merepresentasikan informasi edge secara eksplisit, sehingga konversi ke adjacency list hanya membutuhkan pembacaan dan pemformatan ulang data yang sudah ada.

Adjacency list adalah representasi graph yang menyimpan, untuk setiap vertex, daftar vertex lain yang dapat dicapai langsung beserta bobot edge-nya. Berbeda dengan adjacency matrix yang mengalokasikan ruang untuk semua kemungkinan pasangan vertex, adjacency list hanya menyimpan edge yang benar-benar ada, sehingga lebih hemat memori untuk graph

yang jarang (sparse graph). Pada soal ini, adjacency list direpresentasikan dalam bentuk teks terformat di mana setiap baris menampilkan satu vertex sumber diikuti daftar pasangan (vertex tujuan, bobot).

Teknik yang digunakan adalah iterasi baris per baris pada matrix. Cara kerjanya memanfaatkan fakta bahwa setiap baris ke- i pada adjacency matrix merepresentasikan seluruh edge keluar dari vertex i . Untuk setiap baris, kita iterasi seluruh kolom j dan memeriksa apakah $\text{mat}[i][j]$ bernilai lebih dari 0. Jika ya, berarti terdapat edge dari vertex i ke vertex j dengan bobot $\text{mat}[i][j]$, sehingga pasangan ($\text{label}[j]$, $\text{mat}[i][j]$) dicetak sebagai bagian dari adjacency list vertex i . Urutan pencetakan sudah otomatis mengikuti urutan label pada input karena kolom diiterasi dari kiri ke kanan sesuai urutan vertex yang diberikan. Kasus khusus ditangani ketika seluruh nilai pada baris i bernilai 0, yang berarti vertex tersebut tidak memiliki edge keluar sama sekali, sehingga dicetak tanda - sebagai gantinya. Asumsi yang dibuat dalam solusi ini adalah bahwa nilai 0 pada matrix secara eksklusif berarti tidak ada edge, dan tidak mungkin ada edge dengan bobot 0. Hal ini sesuai dengan batasan soal yang menyatakan bobot edge selalu bernilai positif ($A_{ij} > 0$ untuk edge yang ada). Selain itu diasumsikan diagonal utama selalu bernilai 0 sesuai batasan soal, sehingga tidak perlu penanganan khusus untuk kemungkinan self-loop.

Analisis kompleksitas waktu solusi ini adalah $O(N^2)$ karena kita harus mengiterasi seluruh elemen matrix, setiap baris memiliki N kolom dan terdapat N baris, sehingga total iterasi adalah $N \times N$. Tidak ada cara yang lebih efisien karena kita memang perlu memeriksa setiap sel untuk menentukan ada tidaknya edge. Kompleksitas ruang adalah $O(N^2)$ untuk menyimpan matrix. Dengan batasan $N \leq 10$, total elemen yang diproses maksimal hanya 100, sehingga solusi berjalan sangat cepat dan tidak mendekati batas waktu 1 detik maupun batas memori 64 MB.

Bukti kebenaran solusi ini dapat dilihat dari korespondensi langsung antara struktur matrix dan adjacency list. Karena adjacency matrix pada dasarnya sudah mengkodekan seluruh informasi edge secara eksplisit, proses konversi ke adjacency list hanyalah pembacaan ulang data yang sama dalam format berbeda. Pada contoh kasus, baris vertex A memiliki nilai positif pada kolom B (nilai 4) dan kolom C (nilai 2), sehingga adjacency list untuk A adalah (B,4), (C,2), sesuai output yang diharapkan. Baris vertex E seluruhnya bernilai 0, sehingga dicetak tanda -, yang juga konsisten dengan output yang diharapkan. Karena setiap sel

diperiksa tepat satu kali dan kondisi pencetakan hanya bergantung pada nilai sel tersebut, tidak ada kemungkinan edge terlewat maupun edge yang salah dicetak.

SOAL 3 : BFS: JARAK TERPENDEK KELUAR LABIRIN

Time Limit: 1.0 s Memory Limit: 64 MB
--

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Table 3 Source Code BFS Terpendek Labirin

1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int main() {
5	ios::sync_with_stdio(false);
6	cin.tie(nullptr);
7	
8	int R, C;
9	cin >> R >> C;
10	
11	vector<vector<int>> grid(R, vector<int>(C));
12	for (int i = 0; i < R; i++)
13	for (int j = 0; j < C; j++)
14	cin >> grid[i][j];
15	
16	int sr, sc, fr, fc;
17	cin >> sr >> sc >> fr >> fc;
18	
19	vector<vector<int>> dist(R, vector<int>(C, -1));
20	queue<pair<int,int>> q;
21	dist[sr][sc] = 0;
22	q.push({sr, sc});
23	
24	int dx[] = {-1, 1, 0, 0};
25	int dy[] = {0, 0, -1, 1};
26	
27	while (!q.empty()) {
28	auto [x, y] = q.front();
29	q.pop();
30	
31	if (x == fr && y == fc) break;
32	

```

33         for (int d = 0; d < 4; d++) {
34             int nx = x + dx[d];
35             int ny = y + dy[d];
36             if (nx >= 0 && nx < R && ny >= 0 && ny < C
37                 && grid[nx][ny] == 0 && dist[nx][ny] ==
-1) {
38                 dist[nx][ny] = dist[x][y] + 1;
39                 q.push({nx, ny});
40             }
41         }
42     }
43
44     cout << dist[fr][fc] << "\n";
45     return 0;
46 }

```

B. Output Program

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> g++ -std=c++17 -o soal3 soal3.cpp
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a>

```

Gambar 3 Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Soal ini meminta kita mencari jumlah langkah minimum dari posisi awal menuju posisi tujuan pada sebuah grid labirin, di mana pergerakan hanya diperbolehkan ke empat arah (atas, bawah, kiri, kanan) dan hanya pada sel bernilai 0.

Algoritma yang dipilih adalah Breadth-First Search (BFS), dan memang sudah diwajibkan oleh soal. Namun secara konseptual, BFS adalah pilihan yang paling tepat untuk kasus ini karena BFS menjamin bahwa setiap sel pertama kali dikunjungi melalui jalur terpendeknya. Hal ini terjadi karena BFS memproses sel berdasarkan urutan jarak dari titik awal, sel berjarak 1 diproses sebelum sel berjarak 2, sel berjarak 2 sebelum sel berjarak 3, dan seterusnya. Berbeda dengan DFS yang bisa saja menemukan tujuan melalui jalur yang

panjang terlebih dahulu, BFS dijamin menghasilkan jarak minimum tanpa perlu membandingkan semua kemungkinan jalur.

Proses berpikir dimulai dari merepresentasikan labirin sebagai graph implisit, di mana setiap sel adalah node dan setiap pergerakan ke sel tetangga yang valid adalah edge dengan bobot 1. Karena semua edge berbobot sama (1 langkah), BFS sudah cukup dan tidak perlu algoritma seperti Dijkstra. Digunakan array `dist[][]` untuk menyimpan jarak minimum ke setiap sel sekaligus berfungsi sebagai penanda sel yang sudah dikunjungi, sel dengan nilai -1 berarti belum dikunjungi. Setiap kali kita mengunjungi sel baru, jarak sel tersebut diisi dengan jarak sel sebelumnya ditambah 1. Proses berhenti ketika sel tujuan dicapai, dan jawabannya langsung diambil dari `dist[fr][fc]`. Jika BFS selesai tanpa pernah mencapai tujuan, nilai `dist[fr][fc]` tetap -1 yang sekaligus menjadi output untuk kasus tidak ada jalur.

Analisis kompleksitas waktu solusi ini adalah $O(R \times C)$ karena dalam kondisi terburuk seluruh sel pada grid dikunjungi tepat satu kali. Kompleksitas ruang juga $O(R \times C)$ untuk menyimpan array jarak dan antrian BFS. Dengan batasan $R, C \leq 100$, ukuran grid maksimal adalah 10.000 sel, yang sangat ringan untuk diproses dalam batas waktu 1 detik maupun batas memori 64 MB.

Kesimpulannya, BFS adalah algoritma yang paling sesuai untuk mencari jalur terpendek pada grid dengan bobot seragam. Penggunaan array `dist[][]` sekaligus sebagai penanda visited membuat implementasi lebih efisien karena tidak memerlukan array visited terpisah, dan nilai akhir pada sel tujuan langsung memberikan jawaban tanpa perlu rekonstruksi jalur.

Breadth-First Search (BFS) adalah algoritma penelusuran graph yang mengeksplorasi node berdasarkan urutan jaraknya dari titik awal. BFS menggunakan struktur data antrian (queue) yang bersifat FIFO (First In First Out) untuk memastikan node yang lebih dekat dari titik awal selalu diproses terlebih dahulu sebelum node yang lebih jauh. Sifat inilah yang menjadi dasar mengapa BFS menjamin ditemukannya jalur terpendek pada graph dengan bobot seragam, ketika suatu node pertama kali dikunjungi oleh BFS, jalur yang digunakan untuk mencapainya dijamin merupakan jalur dengan jumlah langkah paling sedikit.

Teknik yang digunakan adalah memodelkan labirin sebagai graph implisit, di mana setiap sel grid merupakan sebuah node dan setiap pergerakan ke sel tetangga yang valid merupakan edge dengan bobot 1. Karena seluruh edge memiliki bobot yang sama, BFS sudah cukup untuk menemukan jalur terpendek tanpa perlu algoritma yang lebih kompleks seperti

Dijkstra. Cara kerjanya dimulai dengan memasukkan sel awal ke dalam antrian dengan jarak 0. Setiap iterasi mengambil satu sel dari depan antrian, lalu memeriksa keempat tetangganya (atas, bawah, kiri, kanan). Tetangga yang valid, yaitu berada di dalam batas grid, bernilai 0, dan belum pernah dikunjungi, dimasukkan ke antrian dengan jarak sel saat ini ditambah 1. Proses ini berlanjut hingga sel tujuan dicapai atau antrian habis. Digunakan array `dist[][]` untuk menyimpan jarak minimum ke setiap sel sekaligus berfungsi sebagai penanda visited, sel yang belum dikunjungi diberi nilai -1, sehingga tidak diperlukan array visited terpisah.

Asumsi yang dibuat dalam solusi ini adalah bahwa setiap langkah memiliki biaya yang sama yaitu 1, sehingga BFS sudah cukup tanpa perlu algoritma shortest path berbasis priority queue. Selain itu diasumsikan pergerakan diagonal tidak diperbolehkan, sesuai batasan soal, sehingga setiap sel memiliki maksimal 4 tetangga. Sel start dan finish diasumsikan selalu bernilai 0 sesuai jaminan pada soal, sehingga tidak perlu penanganan kasus di mana titik awal atau tujuan adalah dinding.

Analisis kompleksitas waktu solusi ini adalah $O(R \times C)$ karena dalam kondisi terburuk seluruh sel pada grid dikunjungi tepat satu kali oleh BFS. Setiap sel hanya dimasukkan ke antrian satu kali berkat penandaan visited melalui array `dist[][]`, sehingga tidak ada pemrosesan duplikat. Kompleksitas ruang juga $O(R \times C)$ untuk menyimpan array `dist[][]` dan isi antrian BFS dalam kondisi terburuk. Dengan batasan $R, C \leq 100$, ukuran grid maksimal adalah 10.000 sel, yang sangat ringan untuk diproses dalam batas waktu 1 detik dan batas memori 64 MB.

Bukti kebenaran solusi ini bertumpu pada invariant BFS, yaitu bahwa ketika suatu sel pertama kali dikeluarkan dari antrian, nilai `dist` pada sel tersebut sudah pasti merupakan jarak terpendek dari sel awal. Hal ini dapat dibuktikan secara induktif, sel awal memiliki jarak 0 yang sudah pasti benar, dan setiap sel berikutnya yang dimasukkan ke antrian mendapat nilai jarak dari sel yang menambahkannya ditambah 1, yang karena sifat FIFO antrian dijamin merupakan jarak terpendek. Pada contoh kasus, BFS berhasil menemukan jalur dari (0,0) ke (7,9) dengan 16 langkah. Kasus tidak ada jalur ditangani secara otomatis karena jika sel tujuan tidak pernah dicapai, nilai `dist[fr][fc]` tetap -1 yang langsung dicetak sebagai output sesuai format yang diminta.

SOAL 4 : DFS: MENGHITUNG SEMUA JALUR KELUAR LABIRIN

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$SR\ SC$

$FR\ FC$

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Table 4 Source Code DFS Semua Labirin

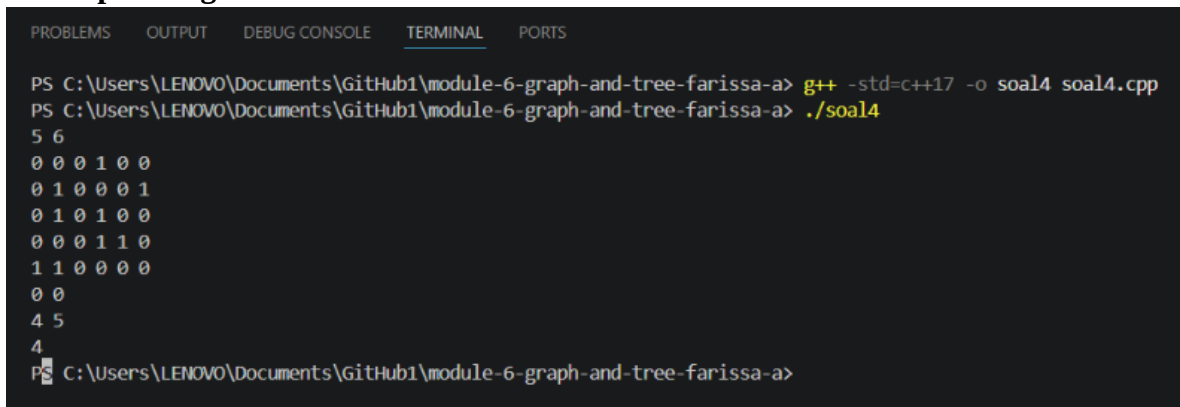
1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int R, C, fr, fc;
5	vector<vector<int>> grid;
6	vector<vector<bool>> visited;
7	int pathCount = 0;
8	
9	int dx[] = {-1, 1, 0, 0};
10	int dy[] = {0, 0, -1, 1};
11	
12	void dfs(int x, int y) {
13	if (x == fr && y == fc) {
14	pathCount++;
15	return;
16	}
17	
18	for (int d = 0; d < 4; d++) {
19	int nx = x + dx[d];
20	int ny = y + dy[d];
21	if (nx >= 0 && nx < R && ny >= 0 && ny < C
22	&& grid[nx][ny] == 0 && !visited[nx][ny]) {
23	visited[nx][ny] = true;
24	dfs(nx, ny);
25	visited[nx][ny] = false;
26	}
27	}
28	}
29	
30	int main() {
31	ios::sync_with_stdio(false);
32	cin.tie(nullptr);
33	
34	cin >> R >> C;
35	grid.assign(R, vector<int>(C));
36	for (int i = 0; i < R; i++)
37	for (int j = 0; j < C; j++)
38	cin >> grid[i][j];
39	
40	int sr, sc;
41	cin >> sr >> sc >> fr >> fc;

```

42
43     visited.assign(R, vector<bool>(C, false));
44     visited[sr][sc] = true;
45     dfs(sr, sc);
46
47     cout << pathCount << "\n";
48     return 0;
49 }

```

B. Output Program



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> g++ -std=c++17 -o soal4 soal4.cpp
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a>

```

Gambar 4 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Soal ini meminta kita menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan pada grid labirin, di mana setiap jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.

Algoritma yang dipilih adalah Depth-First Search (DFS) rekursif dengan backtracking, dan memang sudah diwajibkan oleh soal. Secara konseptual, DFS dengan backtracking adalah satu-satunya pendekatan yang tepat untuk masalah ini karena kita tidak hanya mencari satu jalur terpendek, melainkan harus menelusuri dan menghitung seluruh kemungkinan jalur yang valid. BFS tidak cocok di sini karena BFS dirancang untuk menemukan jalur terpendek, bukan untuk mengeksplorasi semua kemungkinan jalur secara exhaustive. DFS secara alami menelusuri satu jalur hingga ujung sebelum mencoba jalur lain, yang sesuai dengan kebutuhan pencacahan jalur sederhana.

Proses berpikir dimulai dari memahami bahwa backtracking adalah kunci utama solusi ini. Ketika DFS memasuki sebuah sel, sel tersebut ditandai sebagai visited agar tidak dikunjungi lagi dalam jalur yang sama. Setelah seluruh kemungkinan dari sel tersebut selesai dieksplorasi, tanda visited-nya dibatalkan Kembali, inilah proses backtracking. Pembatalan

ini penting karena sel yang sama boleh dikunjungi pada jalur yang berbeda, hanya tidak boleh dikunjungi dua kali dalam satu jalur yang sama. Setiap kali DFS mencapai sel tujuan, counter pathCount diinkremen sebesar 1, lalu fungsi langsung kembali tanpa melanjutkan eksplorasi dari sel tujuan tersebut. Kondisi batas eksplorasi adalah ketika posisi keluar dari grid, mengenai dinding (nilai 1), atau sel sudah ditandai visited.

Analisis kompleksitas waktu solusi ini dalam kondisi terburuk adalah $O(4^{(R \times C)})$ karena setiap sel memiliki hingga 4 pilihan arah dan kita menelusuri semua kemungkinan jalur. Namun dalam praktiknya jauh lebih kecil karena banyak sel yang terblokir dinding atau sudah visited yang memangkas ruang pencarian secara signifikan. Batasan soal pun sudah mempertimbangkan hal ini dengan membatasi $R, C \leq 7$ dan menjamin jumlah jalur valid tidak lebih dari 100.000, sehingga solusi tetap dapat selesai dalam batas waktu 2 detik. Kompleksitas ruang adalah $O(R \times C)$ untuk array visited dan stack rekursi.

Kesimpulannya, DFS rekursif dengan backtracking adalah pendekatan yang paling natural dan tepat untuk masalah pencacahan jalur sederhana. Mekanisme backtracking memastikan bahwa setiap jalur dieksplorasi secara independen tanpa saling mengganggu, sementara penandaan visited mencegah pengulangan sel dalam satu jalur yang sama. Kombinasi keduanya memungkinkan kita menghitung seluruh jalur valid secara sistematis dan akurat.

Depth-First Search (DFS) adalah algoritma penelusuran graph yang mengeksplorasi sejauh mungkin ke dalam satu cabang sebelum mundur dan mencoba cabang lainnya. DFS menggunakan struktur data stack, yang dalam implementasi rekursif diwakili oleh call stack sistem secara implisit. Berbeda dengan BFS yang mengeksplorasi node secara melebar berdasarkan jarak, DFS menelusuri satu jalur hingga ujungnya terlebih dahulu sebelum kembali (backtrack) dan mencoba jalur lain. Sifat inilah yang membuat DFS cocok untuk masalah yang membutuhkan eksplorasi exhaustive terhadap semua kemungkinan jalur, bukan hanya jalur terpendek.

Backtracking adalah teknik yang dipadukan dengan DFS untuk memungkinkan eksplorasi semua kemungkinan secara sistematis. Cara kerjanya adalah dengan menandai setiap sel sebagai visited ketika DFS memasukinya, lalu membatalkan tanda tersebut setelah seluruh eksplorasi dari sel itu selesai dilakukan. Pembatalan tanda visited ini adalah inti dari backtracking, dengan membatalkan tanda, sel tersebut menjadi tersedia kembali untuk digunakan pada jalur lain yang berbeda. Setiap kali DFS mencapai sel tujuan, counter

pathCount diinkremen sebesar 1 dan fungsi langsung kembali tanpa melanjutkan eksplorasi lebih jauh dari sel tujuan. Eksplorasi dihentikan pada suatu cabang ketika posisi keluar dari batas grid, mengenai sel bernilai 1 (dinding), atau sel sudah ditandai visited dalam jalur saat ini.

Asumsi yang dibuat dalam solusi ini adalah bahwa sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama, sesuai definisi pada soal. Oleh karena itu penandaan visited dilakukan per jalur, bukan secara global permanen seperti pada BFS. Sel yang sama boleh dikunjungi pada jalur yang berbeda, sehingga tanda visited harus dibatalkan setiap kali backtracking terjadi. Selain itu diasumsikan bahwa sel start dan finish selalu bernilai 0, dan pergerakan hanya diperbolehkan ke empat arah tanpa diagonal, sesuai batasan soal.

Analisis kompleksitas waktu solusi ini dalam kondisi terburuk adalah $O(4^{(R \times C)})$ karena pada setiap sel terdapat hingga 4 pilihan arah dan kita menelusuri semua kemungkinan kombinasi jalur secara exhaustive. Namun dalam praktiknya kompleksitas jauh lebih kecil karena keberadaan dinding dan penandaan visited memangkas ruang pencarian secara signifikan, banyak cabang yang dihentikan lebih awal sebelum mencapai kedalaman maksimum. Batasan soal sudah mempertimbangkan hal ini dengan membatasi ukuran grid $R, C \leq 7$ serta menjamin jumlah jalur valid tidak lebih dari 100.000, sehingga solusi tetap dapat selesai dalam batas waktu 2 detik. Kompleksitas ruang adalah $O(R \times C)$ untuk array visited dan kedalaman maksimal call stack rekursi yang setara dengan panjang jalur terpanjang yang mungkin.

Bukti kebenaran solusi ini bertumpu pada dua properti utama. Pertama, properti kelengkapan, DFS dengan backtracking menjamin bahwa setiap jalur sederhana yang mungkin pasti dieksplorasi karena setiap kombinasi sel yang belum visited selalu dicoba secara sistematis. Tidak ada jalur valid yang terlewat karena backtracking memastikan sel yang sudah digunakan di jalur sebelumnya selalu tersedia kembali untuk jalur berikutnya. Kedua, properti keunikan, setiap jalur dihitung tepat satu kali karena DFS selalu menelusuri jalur dalam urutan yang deterministik (atas, bawah, kiri, kanan) dan penandaan visited dalam satu jalur mencegah sel yang sama dikunjungi dua kali dalam jalur tersebut, sehingga tidak ada jalur yang terhitung duplikat. Pada contoh kasus, DFS berhasil menemukan dan menghitung 4 jalur sederhana dari (0,0) ke (4,5), yang konsisten dengan output yang diharapkan.

SOAL 5 : TREE TRAVERSAL

Time Limit: 1.0 s Memory Limit: 64 MB
--

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTREE terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTREE.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

• $0 \leq N \leq 1000$

• $-10000000000 \leq A_i \leq 10000000000$

• $1 \leq Q \leq 20$

• T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan. Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Table 5 Source Code Tree Traversal

1	#include <bits/stdc++.h>
2	#include "RedBlackTree.h"
3	using namespace std;
4	
5	void preorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& result) {
6	if (node == nil) return;
7	result.push_back(node->key);
8	preorder(node->left, nil, result);
9	preorder(node->right, nil, result);
10	}
11	
12	void inorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& result) {
13	if (node == nil) return;
14	inorder(node->left, nil, result);
15	result.push_back(node->key);
16	inorder(node->right, nil, result);
17	}
18	
19	void postorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& result) {
20	if (node == nil) return;
21	postorder(node->left, nil, result);
22	postorder(node->right, nil, result);
23	result.push_back(node->key);
24	}
25	
26	void printTraversal(const string& label, const vector<int>& v) {
27	cout << label << " :";
28	for (int x : v) cout << " " << x;
29	cout << "\n";
30	}
31	
32	int main() {
33	ios::sync_with_stdio(false);
34	cin.tie(nullptr);
35	
36	int N;
37	cin >> N;
38	
39	RedBlackTree rbt;
40	for (int i = 0; i < N; i++) {
41	int a;


```

42         cin >> a;
43         if (!rbt.contains(a)) {
44             rbt.insert(a);
45         }
46     }
47
48     int Q;
49     cin >> Q;
50
51     while (Q--) {
52         string query;
53         cin >> query;
54
55         if (rbt.empty()) {
56             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
57             continue;
58         }
59
60         const RedBlackTree::Node* r = rbt.root();
61         const RedBlackTree::Node* nil = rbt.nil();
62
63         if (query == "PREORDER") {
64             vector<int> res;
65             preorder(r, nil, res);
66             printTraversal("[Preorder]", res);
67         } else if (query == "INORDER") {
68             vector<int> res;
69             inorder(r, nil, res);
70             printTraversal("[Inorder]", res);
71         } else if (query == "POSTORDER") {
72             vector<int> res;
73             postorder(r, nil, res);
74             printTraversal("[Postorder]", res);
75         } else if (query == "ALL") {
76             vector<int> pre, in, post;
77             preorder(r, nil, pre);
78             inorder(r, nil, in);
79             postorder(r, nil, post);
80             printTraversal("[Preorder]", pre);
81             printTraversal("[Inorder]", in);
82             printTraversal("[Postorder]", post);
83         }
84     }
85
86     return 0;
87 }

```

B. Output Program



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> g++ -std=c++17 -o soal5 soal5.cpp RedBlackTree.cpp
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\Users\LENOVO\Documents\GitHub1\module-6-graph-and-tree-farissa-a> |
```

Gambar 5 Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Soal ini meminta kita memasukkan N bilangan bulat ke dalam Red-Black Tree (RBT) secara berurutan, mengabaikan duplikat, lalu menjawab Q query traversal yang masing-masing meminta hasil Preorder, Inorder, Postorder, atau ketiganya sekaligus (ALL).

Algoritma yang dipilih adalah memanfaatkan implementasi Red-Black Tree yang sudah disediakan, kemudian mengimplementasikan ketiga fungsi traversal secara rekursif di atasnya. Red-Black Tree dipilih sebagai struktur data karena sifatnya sebagai self-balancing BST menjamin tinggi pohon selalu $O(\log N)$, sehingga operasi insert dan contains berjalan efisien. Penggunaan RBT dibandingkan BST biasa juga memastikan pohon tidak terdegenerasi menjadi linked list pada kasus input yang sudah terurut, yang akan membuat operasi menjadi $O(N)$ per insert.

Proses berpikir dimulai dari memahami struktur RBT yang disediakan. RBT menggunakan sentinel node nilNode sebagai penanda akhir pohon, menggantikan pointer nullptr yang biasa digunakan pada BST konvensional. Karena itu, kondisi berhenti pada setiap fungsi traversal bukan mengecek `node == nullptr`, melainkan mengecek `node == nil` menggunakan pointer yang didapat dari `rbt.nil()`. Tanpa memahami detail implementasi ini, traversal akan salah karena terus masuk ke sentinel node secara tidak terbatas. Untuk menangani duplikat, sebelum memanggil `insert()` dilakukan pengecekan dengan `contains()` terlebih dahulu, jika nilai sudah ada maka dilewati. Kasus edge ketika semua nilai adalah duplikat sehingga tree kosong juga ditangani dengan mencetak pesan khusus sesuai format yang diminta.

Ketiga fungsi traversal diimplementasikan secara rekursif mengikuti definisi standarnya. Preorder mengunjungi root terlebih dahulu, lalu subtree kiri, lalu subtree kanan, cocok untuk merekonstruksi struktur pohon. Inorder mengunjungi subtree kiri, lalu root, lalu subtree kanan, pada BST dan RBT, inorder selalu menghasilkan urutan nilai dari terkecil ke terbesar, yang dapat digunakan untuk memverifikasi kebenaran struktur pohon. Postorder mengunjungi subtree kiri, lalu subtree kanan, lalu root, umumnya digunakan untuk operasi penghapusan pohon secara aman. Untuk query ALL, ketiga traversal dipanggil secara berurutan dan hasilnya dicetak dalam urutan Preorder, Inorder, lalu Postorder.

Analisis kompleksitas waktu untuk proses insert N bilangan adalah $O(N \log N)$ karena setiap insert pada RBT membutuhkan $O(\log N)$ termasuk proses penyeimbangan `fixInsert`. Setiap query traversal membutuhkan $O(N)$ karena setiap node dikunjungi tepat satu kali. Dengan batasan $N \leq 1000$ dan $Q \leq 20$, total kompleksitas adalah $O(N \log N + Q \times N)$ yang sangat ringan untuk batas waktu 1 detik. Kompleksitas ruang adalah $O(N)$ untuk menyimpan seluruh node pada pohon ditambah $O(\log N)$ untuk stack rekursi traversal.

Kesimpulannya, kunci keberhasilan solusi ini terletak pada pemahaman mendalam terhadap implementasi RBT yang disediakan, khususnya penggunaan sentinel node `nilNode` sebagai penanda batas pohon. Dengan memahami detail tersebut, implementasi traversal rekursif menjadi straightforward dan hasilnya konsisten, terbukti dari hasil Inorder yang selalu menghasilkan urutan terurut, mengonfirmasi bahwa struktur RBT terbentuk dengan benar.

Red-Black Tree (RBT) adalah struktur data self-balancing Binary Search Tree (BST) yang menjamin pohon tetap seimbang setelah setiap operasi insert maupun delete. Keseimbangan ini dijaga melalui dua mekanisme utama, yaitu pewarnaan node (setiap node diberi warna merah atau hitam) dan rotasi (rotasi kiri dan rotasi kanan). RBT menjamin lima properti yang harus selalu terpenuhi: pertama, setiap node berwarna merah atau hitam. Kedua, root selalu berwarna hitam. Ketiga, setiap node daun (sentinel/`nil`) berwarna hitam. Keempat, setiap node merah hanya boleh memiliki anak yang berwarna hitam. Kelima, setiap jalur dari root ke node daun mengandung jumlah node hitam yang sama. Kombinasi kelima properti ini menjamin tinggi pohon selalu $O(\log N)$, sehingga operasi insert, delete, dan search selalu berjalan dalam waktu $O(\log N)$ terlepas dari urutan data yang dimasukkan.

Tree traversal adalah teknik untuk mengunjungi seluruh node pada pohon dalam urutan tertentu. Terdapat tiga jenis traversal standar yang diminta pada soal ini. Preorder

mengunjungi node dengan urutan root terlebih dahulu, kemudian subtree kiri, lalu subtree kanan, teknik ini berguna untuk merekonstruksi struktur pohon karena root selalu dicetak sebelum anak-anaknya. Inorder mengunjungi node dengan urutan subtree kiri terlebih dahulu, kemudian root, lalu subtree kanan, pada BST dan RBT, inorder selalu menghasilkan urutan nilai dari terkecil ke terbesar, sehingga dapat digunakan untuk memverifikasi kebenaran struktur pohon. Postorder mengunjungi node dengan urutan subtree kiri terlebih dahulu, kemudian subtree kanan, lalu root, teknik ini umumnya digunakan untuk operasi yang membutuhkan pemrosesan anak sebelum induknya, seperti penghapusan pohon secara aman. Ketiga traversal diimplementasikan secara rekursif karena struktur rekursif dari pohon biner secara alami sesuai dengan cara kerja rekursi.

Asumsi yang dibuat dalam solusi ini adalah bahwa implementasi RBT yang disediakan menggunakan sentinel node `nilNode` sebagai penanda batas pohon, menggantikan pointer `nullptr` yang biasa digunakan pada BST konvensional. Oleh karena itu kondisi berhenti pada setiap fungsi traversal bukan mengecek `node == nullptr`, melainkan mengecek `node == nil` menggunakan pointer yang didapat dari `rbt.nil()`. Asumsi ini krusial karena tanpanya traversal akan terus masuk ke sentinel node secara tidak terbatas dan menyebabkan infinite recursion. Selain itu diasumsikan bahwa nilai duplikat diabaikan sepenuhnya dan tidak dimasukkan ke dalam pohon, sesuai ketentuan soal, sehingga setiap nilai pada RBT dijamin unik.

Analisis kompleksitas waktu untuk proses insert N bilangan adalah $O(N \log N)$ karena setiap operasi insert pada RBT membutuhkan $O(\log N)$ untuk penelusuran posisi yang tepat ditambah $O(\log N)$ untuk proses penyeimbangan melalui `fixInsert` yang melibatkan recoloring dan rotasi. Pengecekan duplikat menggunakan `contains()` juga membutuhkan $O(\log N)$ per elemen, sehingga total preprocessing tetap $O(N \log N)$. Setiap query traversal membutuhkan $O(N)$ karena setiap node dikunjungi tepat satu kali tanpa pengulangan. Dengan batasan $N \leq 1000$ dan $Q \leq 20$, total kompleksitas keseluruhan adalah $O(N \log N + Q \times N)$ yang sangat ringan untuk batas waktu 1 detik. Kompleksitas ruang adalah $O(N)$ untuk menyimpan seluruh node pada pohon ditambah $O(\log N)$ untuk kedalaman maksimal call stack rekursi traversal, yang setara dengan tinggi pohon RBT.

Bukti kebenaran solusi ini dapat dilihat dari dua aspek. Pertama, kebenaran struktur RBT, karena RBT mempertahankan properti BST di mana nilai subtree kiri selalu lebih kecil dari

root dan nilai subtree kanan selalu lebih besar, urutan inorder yang dihasilkan dijamin terurut dari kecil ke besar. Hal ini terbukti pada contoh kasus di mana inorder menghasilkan 20 30 40 50 60 70 80, yang merupakan urutan terurut dari seluruh nilai yang dimasukkan. Kedua, kebenaran traversal, karena setiap fungsi traversal mengecek kondisi node == nil sebelum memproses node, tidak ada node sentinel yang ikut tercetak dan tidak ada node data yang terlewat. Pada contoh kasus, preorder menghasilkan 50 30 20 40 70 60 80 yang menunjukkan 50 sebagai root, subtree kiri berisi 30 20 40, dan subtree kanan berisi 70 60 80, konsisten dengan struktur RBT yang terbentuk dari urutan insert yang diberikan dan seluruhnya sesuai dengan output yang diharapkan.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-farissa-a.git>